

Autonomous Testing

SE Radio 674: Vilhelm von Ehrenheim on Autonomous Testing

Contenido

Sobre el ponente: Vilhelm von Ehrenheim.....	1
Qué es el Autonomous Testing.....	1
Por qué usar pruebas autónomas	2
Tipos de pruebas autónomas.....	2
Self-healing.....	2
Agentes autónomos.....	2
Qué es un agente de testing autónomo.....	3
Papel del desarrollador humano en testing autónomo	3
Optimización de pruebas	3

Sobre el ponente: Vilhelm von Ehrenheim

El entrevistado del capítulo 674 es Vilhelm von Ehrenheim, cofundador de QA.Tech (de la que es director jefe de IA), una empresa fundada en Estocolmo con una plantilla que ronda los 15 empleados dedicada a construir una plataforma de pruebas autónomas basada en inteligencia artificial para agilizar las pruebas E2E.

Vilhelm cuenta con más de 10 años de experiencia en ciencia de datos y machine learning, habiendo ayudado a construir Mother Brain en EQT (una plataforma basada en IA para la identificación y análisis de oportunidades de inversión). Además, también ha publicado artículos en conferencias como EMNLP¹ y CIKM².

Qué es el Autonomous Testing

Vilhelm define el concepto de pruebas autónomas como, no un concepto rígido, sino una escala o espectro donde se sitúan los diferentes niveles de autonomía a la hora de desarrollar y ejecutar pruebas end-to-end (E2E):

1. **Pruebas manuales:** Nada está automatizado, el desarrollador o tester realiza todas las interacciones con la GUI esperando replicar la misma acción siempre.

¹ EMNLP: Empirical Methods in Natural Language Processing

² CIKM: Conference on Information and Knowledge Management

2. **Uso de automatización:** Uso de herramientas como Playwright, Selenium o Cypress, especializadas en realizar interacciones repetibles sobre una interfaz gráfica.
3. **Pruebas autónomas:** En vez de modificar las pruebas E2E cada vez que se modifica una funcionalidad en scripts *hardcodeados*, enfocar las pruebas en el uso de agentes de IA que validen los objetivos propuestos para la página a desarrollar o ampliar, de forma flexible a cambios como un ser humano haría.

Por qué usar pruebas autónomas

Las pruebas unitarias o pruebas de integración no suelen ser tan sensibles a cambios como las pruebas E2E. En cuanto se modifique una vista de un sitio web, cambios tan pequeños como el formato de una salida, el color de un botón, la disposición del layout, o el tiempo en que aparece un popup puede provocar que todos los tests fallen. Además, es fácil probar de forma aislada componentes o paneles individuales, pero su integración en páginas o sitios web complejos se vuelve una tarea más tediosa y complicada de probar de forma robusta. Las pruebas autónomas vienen a ser la solución al dilema de pasar más tiempo reescribiendo pruebas que código nuevo.

Tipos de pruebas autónomas

Vilhelm identifica 2 formas de emplear pruebas autónomas en el contexto de una interfaz gráfica: actualización automática de casos de prueba mediante el uso de *self-healing*, y toma de decisiones en tiempo real por parte de agentes autónomos.

Self-healing

La IA detecta los cambios en el código de la interfaz gráfica, tales como añadir un botón o cambiar el formato de un texto, y modifica los casos de prueba E2E acorde con estas modificaciones.

Ejemplo: Si la prueba falla porque se cambió el texto de un botón de “Confirmar” a “Aceptar”, la IA modifica el código de los casos de prueba para actualizarlo al texto nuevo.

Agentes autónomos

La IA detecta los cambios en el código de la interfaz gráfica, igual que en el caso anterior, pero en vez de modificar los casos de prueba es capaz de adaptarse a la nueva funcionalidad. Si detecta un nuevo componente o comportamiento se adapta a ello.

Ejemplo: Aunque se cambie el texto del botón, la IA comprende que debe haber un botón encargado de confirmar una acción o enviar un formulario.

Qué es un agente de testing autónomo

Un agente, dentro del contexto de pruebas autónomas, se refiere a un componente software basado en IA que se ejecuta dentro de un entorno controlado, en este caso la interfaz de usuario del navegador conectado a un sitio web desplegado y el código de las pruebas, y mediante el uso de modelos de lenguaje (LLMs) planifica y lleva a cabo acciones concretas sin supervisión humana ni pasos definidos.

Las principales características de un agente de testing autónomo son las siguientes:

1. Un modelo de procesamiento central basado en un LLM encargado de razonar y tomar decisiones.
2. Módulo de observación que permita ver y entender el DOM del navegador y el estado visual del mismo.
3. Almacenamiento en memoria de trayectorias pasadas para aprender de errores y éxitos anteriores.
4. Capacidad de trazar una ruta hacia un objetivo establecido sin contar con una definición paso a paso.
5. Ejecutor que traduce las decisiones de la IA en interacciones con la interfaz: clic, scroll, entrada de texto...

Papel del desarrollador humano en testing autónomo

Igual que cualquier encargado de QA, la IA puede cometer errores en cualquier momento o introducir ciertos sesgos, provocando que se exploten más áreas de la aplicación que otras, dejando algunas prácticamente sin cubrir.

Otra desventaja de la IA es evidente: es una tecnología emergente aún en desarrollo. La IA puede especializarse en resolver problemas relativamente comunes o sencillos, aunque también ayuda en tareas tediosas; sin embargo, puede provocar un efecto de bola de nieve si el problema a resolver está relacionado con problemas lógicos complejos, tecnologías emergentes o problemas sutiles de UX.

En conclusión, el rol del ser humano en QA empleando pruebas autónomas no se limita a ser sustituido ni un mero observador, sino que sigue siendo un rol necesario, ahorrando al desarrollador de reescribir el mismo caso incontables veces, pudiendo dedicar tiempo a otras tareas como hallar flaquezas potenciales y supervisar el trayecto de la IA en su exploración, empleando herramientas como grabaciones completas de sesión o logs de consola.

Optimización de pruebas

Para mejorar el rendimiento de los agentes de inteligencia artificial en el entorno de pruebas autónomas, se puede emplear de métodos o herramientas tales como el fine-tuning, pudiendo ajustar la IA a un contexto o terminología específico del modelo de

negocio de la aplicación a probar, aumentando el grado de experiencia del mismo si es un caso de nicho.

Por otro lado, también se puede ajustar y mejorar los módulos de toma de decisiones y el núcleo de razonamiento basado en LLM; o tal vez recabar más contexto sobre la aplicación, si usa un modelo más extendido (cosas genéricas como redes sociales, e-commerce...).

Herramientas como estas ayudan a reducir el número de *flaky tests* (pruebas que pasan o fallan de forma aleatoria) y el tiempo que se tarda en ejecutar una suite de pruebas.